

# Informatika 3

Prednáška 12: Jazyk C

Michal Mrena



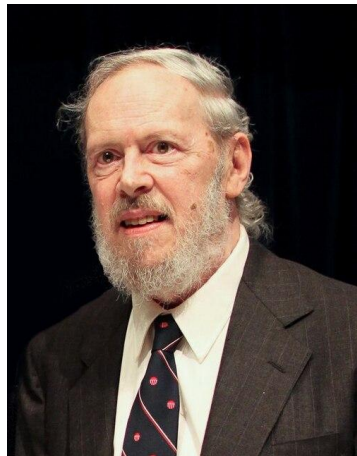
ŽILinskÁ UNIVERZITA V ŽILINE  
Fakulta riadenia  
a informatiky

# Jazyk C

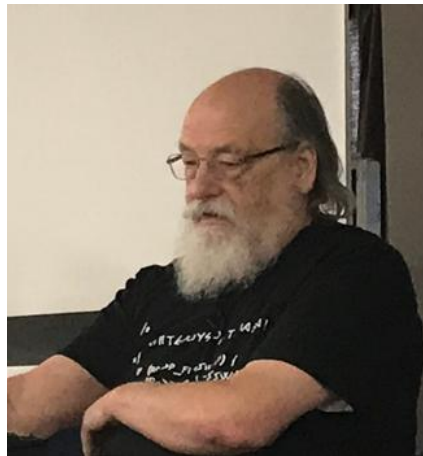


# Jazyk C

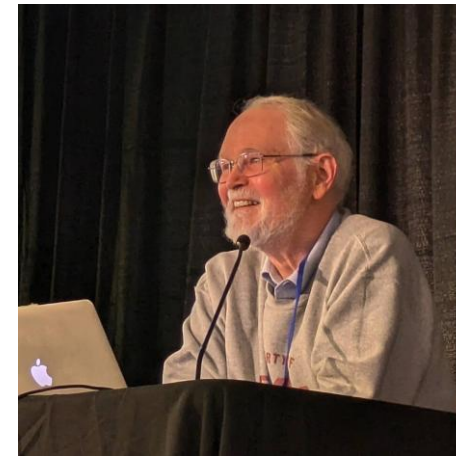
- Bol vytvorený v rokoch 1969 – 1973 v **AT&T Bell Labs** ako nástroj pre programovanie v OS **Unix** a na programovanie jeho jadra



Dennis Ritchie



Ken Thompson



Brian Kernighan

# Jazyk C

- Prívlastky: imperatívny, procedurálny, staticky typovaný, slabo typovaný
- Býva označovaný ako nízkoúrovňový – kód v ňom napísaný sa ľahko mapuje na inštrukcie procesora
- Používa sa pri programovaní operačných systémov ([Linux](#), ...), ovládačov, vstavaných systémov, kompilátorov ([gcc](#), [cpython](#), [lua](#), ...), efektívneho softvéru ([ffmpeg](#), [git](#), [sqlite](#), [nginx](#), [openssl](#)...)
- Je implementovaný pre široké spektrum rôznym platforiem

# Štandardizácia

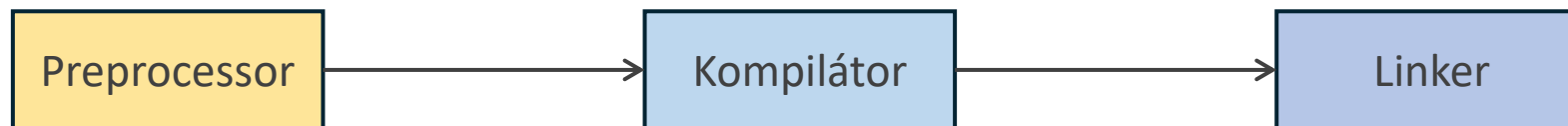
- C89 (ANSI C), C90 (ANSI C akceptovaný ISO), C99, **C11**, C17, C23
- Názvoslovie zahŕňa rok schválenia finálnej podoby štandardu
- V súčasnosti štandard vydáva ISO
  - Za jeho prípravu je zodpovedná komisia – jej členovia pôsobia vo firmách a na univerzitách
  - **Neoficiálna** verzia štandardu je dostupná zdarma
- Štandard je technický dokument, neslúži ako dokumentácia
  - V komunite zaužívaná (ne)oficiálna dokumentácia:  
<https://en.cppreference.com/w/c.html>

# Implementácie

- Jazyk C je pravdepodobne jedným z [najviac implementovaných](#) jazykov
- Najvýznamnejšou implementáciou jazyka je kompilátor [gcc](#), okrem neho napríklad kompilátory [clang](#), [MSVC](#), [tinycc](#) a platformovo špecifické kompilátory [ICC](#) (Intel), [Arm GNU](#)
- Najvýznamnejšou implementáciou štandardnej knižnice je [glibc](#) (GNU), okrem nej knižnice [musl](#), [UCRT](#) (Microsoft)

# Proces prekladu

- Proces prekladu je rovnaký ako proces prekladu jazyka C++ (jazyk C++ vychádza z jazyka C)
- Pozostáva z **oddeleného prekladu** jednotiek prekladu – zdrojové súbory (.c) s vloženými hlavičkovými súbormi (.h)
- Hlavičkové súbory obsahujú deklarácie (a `inline` definície)
- Zdrojové súbory obsahujú definície (vrátane `static` definícií s vnútornou väzbou)
- Jedna entita môže byť deklarovaná ľubovoľný počet krát, definovaná môže byť iba raz



# Prvky jazyka

- Jazyk C je veľmi jednoduchý
- Okrem fundamentálnych typov, výrazov, ... obsahuje:
  - Funkcie (parametre)
  - (globálne, lokálne) Premenné
  - Štruktúry (`struct`, `union`)
  - Vymenovaný typ (`enum`) – slabo typované celočíselné konštanty
  - Slabé typové aliasy (`typedef`)
  - Makrá



# Prvky jazyka



# Štruktúry

```
#include <time.h>

struct WeatherRecord {
    char *city;
    time_t date;
    float tempMin;
    float tempMax;
    int elevation;
};
```

Štruktúra (záznam) je zoskupenie niekoľkých premenných v jednej štruktúre.

Hlavičkové súbory štandardnej knižnice používajú (na rozdiel od jazyka C++) príponu **.h**.

Štandardná knižnica jazyka C je obsiahnutá v štandardnej knižnici jazyka C++. Hlavičkový súbor má predponu **c** a nemá príponu. Napríklad (v jazyku C++) **<ctime>**.

Hlavičky exportujú rovnaké typy v mennom priestore **std**. Napríklad **std::time\_t**.

# Alias štruktúry

```
struct WeatherRecord { /* ... */ };  
  
void weatherRecordPrint(struct WeatherRecord *record);
```

Pri práci s typom štruktúra je potrebné použiť kľúčové slovo `struct`.

```
typedef struct {  
    char *city;  
    time_t date;  
    float tempMin;  
    float tempMax;  
    int elevation;  
} WeatherRecord;
```

Typ štruktúry preto často definujeme ako alias anonymnej štruktúry.

```
typedef <old type> <new_type>;
```

Analógia kľúčového slova `using` z jazyka C++.

Definuje (slabý) alias existujúceho typu.

# Inicializácia štruktúry

```
int main(void) {  
    WeatherRecord record;  
    // Na tomto mieste majú  
    // atribúty nedefinované hodnoty  
    record.city = "Žilina";  
    record.date = time(NULL);  
    record.tempMin = 5.6;  
    record.tempMax = 23.0;  
    record.elevation = 358;  
}
```

Po definovaní premennej typu štruktúra majú jej atribúty nedefinované hodnoty.

Inicializovať ich môžeme jednotlivo.

```
int main(void) {  
    WeatherRecord record = {  
        .city = "Žilina",  
        .date = time(NULL),  
        .tempMin = 5.6,  
        .tempMax = 23.0,  
        .elevation = 358  
    };  
}
```

Atribúty môžeme inicializovať aj priamo v mieste definície.

(od C99) Môžeme v inicializácii uviesť názvy atribútov.

# Funkcie

- Štruktúry obsahujú iba **údaje**
- Správanie (**algoritmy**) definujeme osobitne formou funkcií (procedúr)
- (Procedurálny) program je sériou volaní funkcií (procedúr), ktoré manipulujú s údajmi (štruktúrami alebo fundamentálnymi typmi)
- Funkcie **deklarujeme** v hlavičkových súboroch a **definujeme** v zdrojových súboroch

# Funkcie

```
float weatherRecordMinTempDiff(WeatherRecord *r1, WeatherRecord *r2) {  
    return r1->tempMin - r2->tempMin;  
}
```

Funkcie preberajú parametre a môžu vracať hodnotu určeného typu alebo majú návratový typ `void`.

```
void doNothing(void) {  
    puts("I am not doing anything.");  
}
```

(do C23) Ak chceme deklarovať (definovať) funkciu, ktorá nepreberá žiadna parametre, je potrebné v zozname parametrov uviesť kľúčové slovo `void`.

```
void doNothing() {  
    puts("I am also not doing anything.");  
}  
  
int main(void) {  
    doNothing(1, 2, 3, 4);  
}
```

(do C23) V opačnom prípade ju môžeme zavolať s ľubovoľným počtom argumentov.

# Typy

- Základne aritmetické typy `char`, `int` a ich modifikátory `short`, `long` a `signed`, `unsigned` sú rovnaké ako v jazyku C++
- V typových systémoch je však **niekoľko rozdielov**, preto je dôležité pracovať s dokumentáciou
- (do C23) Jazyk C nepoznal typ `uintptr_t` a kľúčové slovo `uintptr`
  - Na vyjadrenie nulového ukazovateľa sa používalo makro `NULL`
- (do C23) Jazyk C nepoznal typ `bool`, namiesto neho sa používal typ `int` (`0` predstavuje hodnotu `false` a `1` hodnotu `true`)
- (od C99) (do C23) Existuje typ `_Bool` a makrá `bool`, `true` a `false` definované v hlavičke `<stdbool.h>`

# Typ bool

```
int weatherRecordIsFreezing(
    WeatherRecord *record
) {
    return record->tempMax < 0;
}

int main(void) {
    WeatherRecord record = {
        .city = "Žilina",
        .date = time(NULL),
        .tempMin = 5.6,
        .tempMax = 23.0,
        .elevation = 358
    };
    int freezing =
        weatherRecordIsFreezing(&record);
    if (freezing == 1) {
        puts("It is freezing out there!");
    }
}
```

(do C99)

```
_Bool weatherRecordIsFreezing(
    WeatherRecord *record
) {
    return record->tempMax < 0;
}

int main(void) {
    WeatherRecord record = {
        .city = "Žilina",
        .date = time(NULL),
        .tempMin = 5.6,
        .tempMax = 23.0,
        .elevation = 358
    };
    _Bool freezing =
        weatherRecordIsFreezing(&record);
    if (freezing == 1) {
        puts("It is freezing out there!");
    }
}
```

(od C99)(do C23)

```
bool weatherRecordIsFreezing(
    WeatherRecord *record
) {
    return record->tempMax < 0;
}

int main(void) {
    WeatherRecord record = {
        .city = "Žilina",
        .date = time(NULL),
        .tempMin = 5.6,
        .tempMax = 23.0,
        .elevation = 358
    };
    bool freezing =
        weatherRecordIsFreezing(&record);
    if (freezing == true) {
        puts("It is freezing out there!");
    }
}
```

(od C99)(do C23)

(od C23)



# Typ ukazovateľ

- Typ ukazovateľ a aritmetika ukazovateľov funguje rovnako ako v jazyku C++
- Jazyk C však nepozná typ odkaz, ukazovatele sú preto jediný spôsob nepriameho prístupu

```
void weatherRecordPrint(WeatherRecord *record) {  
    if (record != NULL) {  
        // ...  
    }  
}  
  
int main(void) {  
    WeatherRecord record = {  
        // ...  
    };  
    weatherRecordPrint(&record);  
}
```

(do C23)

```
void weatherRecordPrint(WeatherRecord *record) {  
    if (record != nullptr) {  
        // ...  
    }  
}  
  
int main(void) {  
    WeatherRecord record = {  
        // ...  
    };  
    weatherRecordPrint(&record);  
}
```

(od C23)

# Makrá

- V jazyku C používame makrá častejšie ako v jazyku C++
- Okrem štandardného použitia direktív `ifndef` .. `endif` a `define` na zabránenie viacnásobnému vloženiu hlavičkového súboru ich používame na:
  - Definovanie celočíselných konštánt
  - Definovanie jednoduchých (pseudo)funkcií (príklad)
  - Generovanie funkcií
  - Definovanie variadických funkcií
- Vo všeobecnosti nie sú makrá typovo bezpečné – ak vieme rovnaké správanie dosiahnuť použitím (`inline`) funkcie je lepšie použiť funkciu

```
#define max(a,b) (((a) > (b)) ? (a) : (b))  
#define min(a,b) (((a) < (b)) ? (a) : (b))
```

# Vymenovaný typ

- Vymenovaný typ v jazyku C je zoskupenie slabotypovaných celočíselných konštánt
- Konštanty sú (do C23) typu `int` a sú voľne dostupné v globálnom mennom priestore
- Pri definícii často používame alias anonymného typu (z rovnakého dôvodu ako pri štruktúre)
- Vymenované hodnoty sú reprezentované celými číslami `0`, `1`, `2`, ...

```
typedef enum {  
    Cloudy,  
    PartlyCloudy,  
    Fog,  
    Sunny  
} Weather;
```

```
bool isSunny(WeatherType t) {  
    return t == Sunny;  
}  
  
int main(void) {  
    WeatherType t = Sunny;  
    if (isSunny(t)) {  
        puts("That is nice.");  
    }  
    int r = Cloudy;  
}
```

# Pole

- Štandardná knižnica jazyka C neobsahuje kontajnery
- Pole je preto hlavnou údajovou štruktúrou
- Polia fungujú rovnako ako polia, ktoré sú súčasťou jazyka C++
- Ich veľkosť musí byť konštanta známa pri preklade
- Pole je uložené v zásobníku, preto je potrebné zohľadňovať jeho veľkosť

```
int main(void) {  
    int numbers[3];  
    numbers[0] = 1;  
    numbers[1] = 10;  
    numbers[2] = 100;  
  
    WeatherRecord records[] = {  
        {"Brezno", time(NULL), 8.9, 14.2, 488},  
        {"Skalnaté Pleso", time(NULL), -1.3, 5.4, 1778},  
        {"Žilina", time(NULL), 5.6, 23.0, 358}  
    };  
}
```

Prvky poľa môžeme inicializovať individuálne alebo priamo pri jeho vytvorení (čím je automaticky určená veľkosť poľa.)  
Inicializáciu môžeme kombinovať s inicializáciou prvkov (štruktúr).

# Pole s variabilnou dĺžkou

- (od C99)(do C11)\* Môžeme vytvoriť aj pole, ktorého veľkosť je známa až za behu programu
- Takéto pole musí byť lokálna premenná
- Kompilátor zabezpečí alokovanie aj dealokovanie pamäte
- Pamäť je zvyčajne alokovaná na zásobníku, štandard to však nevyžaduje

\* (od C11) je ich implementácia voliteľná.

```
void generateRandomRecord(WeatherRecord *r);

void printRandomRecord(int n) {
    WeatherRecord records[n];
    for (int i = 0; i < n; ++i) {
        generateRandomRecord(records + i);
    }
    for (
        WeatherRecord *r = records;
        r < records + n;
        ++r
    ) {
        weatherRecordPrint(*r);
    }
}
```

Na prechod poľom môžeme využiť aritmetiku ukazovateľov.

# Základná práca v jazyku

Práca s pamäťou



# Alokovanie pamäte

Jazyk C umožňuje nízkoúrovňovú priamu prácu s pamäťou – jej alokovanie. Prostredníctvom funkcií malloc, calloc, realloc a free.

Uvedené funkcie pracujú s generickým ukazovateľom `void*`.

```
void *malloc(size_t size);
void *calloc(size_t num, size_t size);
void *realloc(void *ptr, size_t new_size);
void free(void *ptr);
```

```
int main(void) {
    WeatherRecord record = {
        "Brezno", time(NULL), 8.9, 14.2, 488
    };
    void *ptr = &record;
    WeatherRecord *wrPtr = ptr;
    weatherRecordPrint(wrPtr);
}
```

Generický ukazovateľ je implicitne konvertovateľný (v oboch smeroch) na ukazovateľ ľubovoľného typu.

Nie je možné dereferencovať ho (bez typu nie je možné určiť počet bajtov, ktoré sa majú prečítať).  
Nefunguje nad ním aritmetika ukazovateľov.  
Slúži iba na uchovanie adresy.

# Generický ukazovateľ

Generické ukazovatele sa používajú v generickom kóde (napríklad pri práci s vláknami).

Nízkoúrovňová práca s pamäťou spadá do tejto kategórie. Nezaujíma nás typ údajov v pamäti. Ide iba o ukazovateľ na začiatok pamäte a počet bajtov.

```
/**
 * Zopakuje funkciu f n-krát s argumentom arg.
 */
void repeatNTimes(int n, void(*f)(void*), void *arg) {
    for (int i = 0; i < n; ++i) {
        f(arg);
    }
}

void wrPrintDiff(void *arg) {
    WeatherRecord *wr = arg;
    printf("Temp diff is %.2f.\n", wr->tempMax - wr->tempMin);
}

int main(void) {
    WeatherRecord record = {
        "Brezno", time(NULL), 8.9, 14.2, 488
    };
    repeatNTimes(5, wrPrintDiff, &record);
}
```



# Alokovanie pamäte

```
int main(void) {  
    int recordCount = 100;  
    int byteCount = recordCount * sizeof(WeatherRecord);  
    WeatherRecord *records = malloc(byteCount);  
    for (WeatherRecord *r = records; r < records + recordCount; ++r) {  
        weatherRecordInit(r);  
    }  
    free(records);  
}
```

Funkcia `malloc` alokuje súvislý kus pamäte požadovanej veľkosti. Vráti ukazovateľ typu `void*` na začiatok alokovanej pamäte.

Pamäť nijako neinicializuje – práca s ňou (bez následnej inicializácie) vedie na nedefinované správanie.

# Alokovanie pamäte

```
int main(void) {  
    int recordCount = 10;  
    int byteCount = recordCount * sizeof(WeatherRecord);  
    WeatherRecord *records = malloc(byteCount);  
    unsigned char *bs = (unsigned char*)records;  
    for (unsigned char *b = bs; b < bs + byteCount; ++b) {  
        fputc(*b, stdout);  
    }  
    puts("");  
    free(records);  
}
```

Program vypíše hodnoty bajtov interpretované ako znaky.  
Pri každom spustení programu môžu byť iné.

```
6 f+7 ÊÛ k+7 -  
x86_64MINGW_PREFIX=C:/msys64/ucrt64MSYS2_NOSTART=yesMSYSTEM=UCRT64MSYSTEM_CARCH=x86_64MSYSTEM_CHOST=x86_64-  
w64-mingw32MSYSTEM_PREFIX=C:/msys64/ucrt64NUMBER_OF_PROCESSORS=200LDPWD=C:/Users/mrena/OneDrive - Äilinskß  
univerzita v Äiline/_vyucba/inf3/2025/prednaskyORIGINAL_PATH=C:\Windows\System32
```

# Alokovanie pamäte

```
int main(void) {
    int recordCount = 10;
    int byteCount = recordCount * sizeof(WeatherRecord);
    WeatherRecord *records = malloc(byteCount);
    memset(records, 0, byteCount);
    unsigned char *bs = (unsigned char*)records;
    for (unsigned char *b = bs; b < bs + byteCount; ++b) {
        printf("%d", (int)*b);
    }
    puts("");
    free(records);
}
```

Pamäť môžeme vynulovať (všetky bajty nastaviť na hodnotu 0) funkciou `memset`. Pre mnohé typy je toto rozumná predvolená hodnota, s ktorou je bezpečné pracovať.

[illegible]

# Alokovanie vynulovanej pamäte

```
int main(void) {
    int recordCount = 10;
    int byteCount = recordCount * sizeof(WeatherRecord);
    WeatherRecord *records = calloc(recordCount, sizeof(WeatherRecord));
    unsigned char *bs = (unsigned char*)records;
    for (unsigned char *b = bs; b < bs + byteCount; ++b) {
        printf("%d", (int)*b);
    }
    puts("");
    free(records);
}
```

Rovnaký výsledok získame použitím funkcie `calloc`, ktorá alokovanú pamäť vynuluje.

```
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

# Kopírovanie pamäte

```
int main(void) {  
    int recordCount = 10;  
    int byteCount = recordCount * sizeof(WeatherRecord);  
    WeatherRecord *records = createRecords(recordCount);  
    WeatherRecord *newRecords = malloc(byteCount);  
    memcpy(newRecords, records, byteCount);  
    if (memcmp(records, newRecords, byteCount) == 0) {  
        puts("Record arrays are byte-equal!");  
    }  
    free(records);  
    free(newRecords);  
}
```

Na kopírovanie už alokovanej pamäte môžeme použiť funkciu `memcpy`. Na porovnanie dvoch súvislých oblastí pamäte funkciu `memcmp`.

Pri použití funkcie `memcpy` sa oblasti pamäte nesmú prekrývať. Funkcia `memmove` korektne zvláda aj tento prípad.

# Kopírovanie pamäte

```
int main(void) {  
    char buffer[1024] = {0};  
    char secret1[] = "A super-secret string!";  
    char src[] = "A string that we want to copy.";  
    memcpy(buffer, src, 54);  
    for (int i = 0; i < 100; ++i) {  
        fputc(buffer[i], stdout);  
    }  
    fputc('\n', stdout);  
}
```

Nízkoúrovňové kopírovanie pamäte je náchylné na chyby.

*This serious flaw (CVE-2014-0160) is a missing bounds check before a `memcpy` call that uses **non-sanitized** user input as the length parameter. An attacker can trick OpenSSL into allocating a 64KB buffer, **copy more bytes than is necessary** into the buffer, **send that buffer back**, and thus leak the **contents of the victim's memory**, 64KB at a time.*

[OWASP | OpenSSL Heartbleed](#)

# Zmena veľkosti pamäte

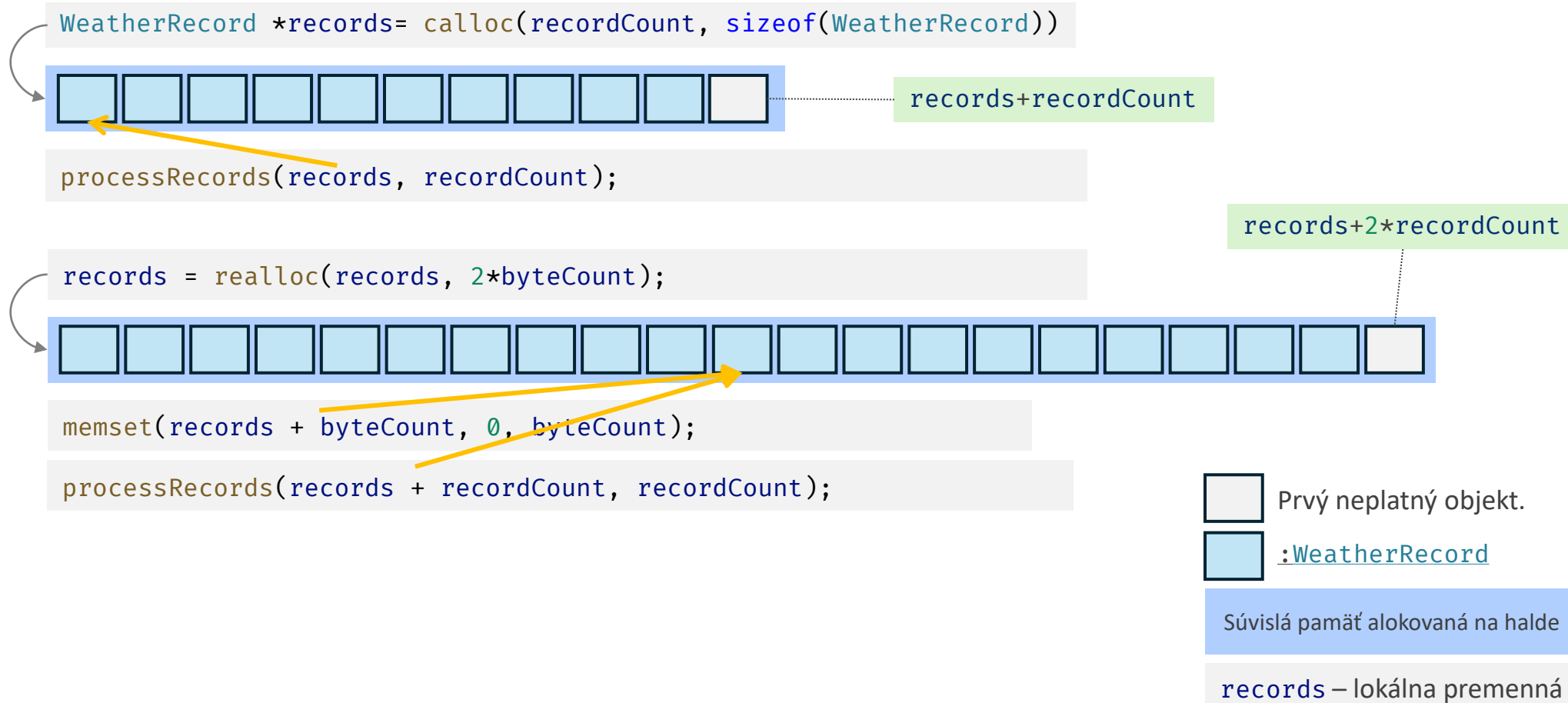
```
int main(void) {  
    int recordCount = 10;  
    int byteCount = recordCount * sizeof(WeatherRecord);  
    WeatherRecord *records = calloc(recordCount, sizeof(WeatherRecord));  
    processRecords(records, recordCount);  
    records = realloc(records, 2*byteCount);  
    memset(records + byteCount, 0, byteCount);  
    processRecords(records + recordCount, recordCount);  
    free(records);  
}
```

Funkcia `realloc` zmení veľkosť už alokovanej pamäte. Môže iba zväčšiť/zmenšiť pamäť na mieste alebo alokovať nový blok – vždy je potrebné spracovať návratovú hodnotu.\*

Ak ako prvý parameter pošleme `NULL`, správa sa ako funkcia `malloc`.

Ešte bezpečnejšie je najprv uložiť návratovú hodnotu do lokálnej premennej.\*

# Zmena veľkosti pamäte





# Uvoľnenie pamäte

```
int main(void) {  
    int recordCount = 10;  
    int byteCount = recordCount * sizeof(WeatherRecord);  
    WeatherRecord *records = createRecords(recordCount);  
    WeatherRecord *newRecords = malloc(byteCount);  
    memcpy(newRecords, records, byteCount);  
    if (memcmp(records, newRecords, byteCount) == 0) {  
        puts("Record arrays are byte-equal!");  
    }  
    free(records);  
    free(newRecords);  
}
```

Na uvoľnenie alokovanej pamäte používame funkciu `free`.  
Ak je jej argumentom `NULL`, definovane nespraví nič.

# Jazyk C++

Nízkoúrovňová práca s pamäťou a objektami



# Jazyk C++

- Uvedené funkcie môžeme použiť aj v jazyku C++
- Pri práci s fundamentálnymi a triviálnymi typmi ich používame prakticky rovnako
- Pri práci s triedami je však potrebné brať do úvahy ich životný cyklus – konštruktory a deštruktory
- Nízkoúrovňový prístup k pamäti nám umožňuje realizovať kroky alokovania pamäte, vytvorenia objektu, zrušenia objektu a uvoľnenia pamäte samostatne

# Jazyk C++

```
class Player {
public:
    Player(int id, std::string name) :
        m_id(id), m_name(std::move(name))
    {
    }

    void introduce() const {
        std::fputs("My name is ", stdout);
        std::fputs(m_name.c_str(), stdout);
        std::puts("");
    }

private:
    int m_id;
    std::string m_name;
};
```

```
#include <cstdio>
#include <cstdlib>
#include <string>
#include <utility>
```

```
1 int main() {
2     Player *p = static_cast<Player*>(
3         std::malloc(sizeof(Player)));
4     new (p) Player(1, "Larry");
5     p->introduce();
6     p->~Player();
7     std::free(p);
8 }
```

(2) Alokovanie pamäte

(4) Vytvorenie objektu operátorom placement-new

(6) Zrušenie objektu explicitným volaním deštruktora

(7) Uvoľnenie pamäte

# Jazyk C++

```
int main() {  
    alignas(Player) std::byte mem[sizeof(Player)];  
    Player *p = reinterpret_cast<Player*>(mem);  
    new (p) Player(1, "Larry");  
    p->introduce();  
    p->~Player();  
}
```

```
#include <new>  
  
int main() {  
    Player *p = static_cast<Player*>(  
        ::operator new(sizeof(Player)));  
    new (p) Player(1, "Larry");  
    p->introduce();  
    p->~Player();  
    ::operator delete(p);  
}
```

Pamäť nemusí pochádzať  
z funkcie `malloc`.

Môžeme použiť vstavané pole s pevnou  
veľkosťou alebo pole `std::array`.  
Je však potrebné myslieť na zarovnanie.

Môžeme sa tiež úplne vyhnúť funkciám  
jazyka C a použiť operátor  
`new` a operátor delete.

# Základná práca v jazyku

Práca s reťazcami



# Práca s reťazcami

- V jazyku C za reťazec považujeme pole znakov ukončené znakom s hodnotou `0` (literál typu `char` reprezentujúci túto hodnotu zapisujeme syntaxou `'\0'`)
- Reťazec `"Hello World!"` by tak bol reprezentovaný poľom o veľkosti `13`
- Pole znakov je zároveň typom literálu (posledný znak `'\0'` je pridaný automaticky)

```
int main(void) {  
    char str[] = "Hello world!";  
    printf("%llu\n", sizeof(str)); // 13  
}
```

# Práca s reťazcami

- Na prácu s reťazcami môžeme použiť funkcie z hlavičky `<string.h>`
- Tieto funkcie predpokladajú, že ako argumenty dostávajú korektné reťazce (ukončené `'\0'`)
- Ak tento predpoklad nie je splnený, ich správanie je nedefinované – ide o častý zdroj chýb

```
int main(void) {  
    char src[] = {'h', 'e', 'l', 'l', 'o'};  
    char dst[6];  
    strcpy(dst, src);  
    puts(dst); // helloHp ε÷  
}
```

Nie je reťazec.

Kopíruje až kým nenájde v pamäti 0

Vypisuje až kým nenájde v pamäti 0.



# Práca s reťazcami

```
char *strncpy(char *dest, const char *src, size_t count);
```

Niektoré funkcie existujú aj v bezpečnejšej verzii, ktorá preberá počet znakov.

```
#define BUFFER_SIZE 1024

void readString(char *dest, size_t count);

int main(void) {
    char buffer1[BUFFER_SIZE] = {0};
    char *buffer2 = calloc(BUFFER_SIZE, sizeof(char));
    readString(buffer1, BUFFER_SIZE);
    readString(buffer2, BUFFER_SIZE);
    if (strcmp(buffer1, buffer2) == 0) {
        puts("They are the same string.");
    }
    free(buffer2);
}
```

Pri práci s reťazcami často používame tzv. buffer. „Pracovné“ pole znakov, v ktorom manipulujeme s reťazcami.

# Základná práca v jazyku

Vstup a výstup



# Vstup a výstup

- Vstupné a výstupné prúdy sú v jazyku C reprezentované štruktúrou `FILE`
- Štandardný vstup a výstup je reprezentovaný globálnymi entitami *tohto* typu
  - `stdout` – štandardný výstup (typicky riadkovo buffrovaný)
  - `stdin` – štandardný vstup (typicky riadkovo buffrovaný)
  - `stderr` – štandardný chybový výstup (typicky nebuffrovaný)
- Funkcie pre vstupné a výstupné operácie sú definované v hlavičke `<stdio.h>`
- Často preberajú
  - prúd (`FILE*`), nad ktorým sa má operácia vykonať (prefix `f`)
  - iba argumenty na výpis/načítanie (bez prefixu, predpokladá sa štandardný vstup/výstup)
  - buffer (`char*`), nad ktorým sa má operácia vykonať (prefix `s`)

# Výstup

```
int main(void) {
    char outs[1024] = {0};
    FILE *outf = fopen("test.txt", "w");
    if (outf == NULL) {
        perror("Failed to open test.txt");
        return EXIT_FAILURE;
    }
    puts("Hello world!");
    fputs("Hello world!", stdout);
    fputs("Hello world!", outf);
    fputc('o', stdout);
    fputc('o', outf);
    fprintf(outf, "Word: %s, len: %d", "hello", 5);
    fprintf(stdout, "Word: %s, len: %d", "hello", 5);
    printf("Word: %s, len: %d", "hello", 5);
    sprintf(outs, "Word: %s, len: %d", "hello", 5);
    fclose(outf);
}
```

Najjednoduchší výstup realizujeme funkciou `fputs` – vypíše reťazec.

Na formátovaný výstup používame funkciu `fprintf`, ktorá preberá formátovací reťazec, ktorý obsahuje zástupné znaky.

# Vstup

```
int main(void) {  
    char buffer[1024] = {0};  
    fgets(buffer, 1024, stdin);  
    fputs("The user entered: ", stdout);  
    fputs(buffer, stdout);  
}
```

```
int main(void) {  
    char buffer[1024] = {0};  
    char *end;  
    fgets(buffer, 1024, stdin);  
    size_t len = strlen(buffer);  
    long x = strtol(buffer, &end, 10);  
    if (end != buffer + len - 1) {  
        fputs("Invalid number: ", stderr);  
        fputs(buffer, stderr);  
    } else {  
        fprintf(stdout, "The user entered %ld\n", x);  
    }  
}
```

Načítanie vstupu je vo všeobecnosti komplikovanejšie.  
Na jednoduché načítanie reťazca môžeme použiť funkciu `fgets`.

Načítaný reťazec môžeme ďalej spracovávať – konvertovať na iné typy.

Na konverziu na čísla môžeme použiť funkciou `strtol` a podobné.

# Vstup

```
int main(void) {
    FILE *f = fopen("input.txt", "r");
    char buffer[1024] = {0};
    int argsParsed;
    argsParsed = fscanf(f, "%1023s\n", buffer);
    fprintf(stdout, "The word is %s\n", buffer);
    if (argsParsed != 1) {
        fputs("Invalid input!\n", stderr);
    }

    float x, y, z;
    argsParsed = fscanf(f, "(%f, %f, %f)", &x, &y, &z);
    fprintf(stdout, "The point is (%f,%f,%f)\n", x, y, z);
    if (argsParsed != 3) {
        fputs("Invalid input!\n", stderr);
    }
    fclose(f);
}
```

here  
(1,2,3)

Na jednoduchšie parsovanie vstupu  
môžeme použiť funkciu `fscanf`  
a formát vstupu špecifikovať  
formátovacím reťazcom.

# Zhrnutie

- V jazyku C programujeme procedurálne
- Program má *blízko* k binárnemu kódu, ktorý vykonáva CPU
- Nízkoúrovňová práca s pamäťou nám dáva veľa voľnosti
- Zároveň vytvára veľký priestor pre chyby

# Ďakujem za pozornosť

Priestor na otázky

